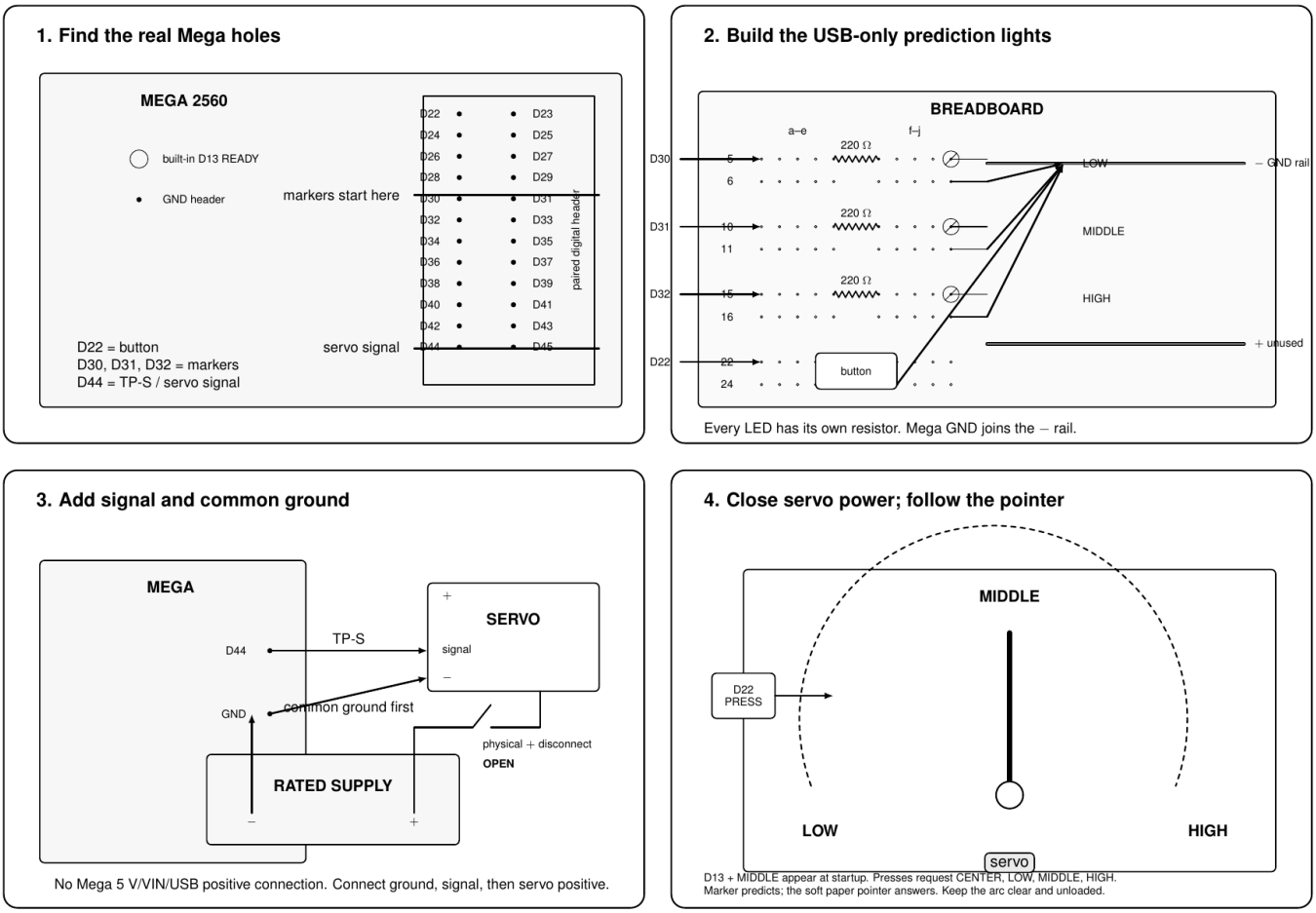
Give a Paper Pointer a Mind

Build the marker lights, predict the move, then watch it happen

Every press earns a visible result: center, low, middle, high.

Lesson	017 — component	**Board**	Arduino Mega 2560
Payoff	a paper pointer follows three command lights	**Control**	button on D22
Energy	E2, exact separately powered servo	**Run**	two minutes, then quiet

The four-picture build



Four pencil plates show the actual build order. Plate 1 makes a paper pointer and low/middle/high card with all power off. Plate 2 shows the Mega paired D30/D31, D32/D33, D34/D35 header geometry and three separately resistor-limited LEDs on a breadboard. Plate 3 joins Mega D44 to servo signal and Mega GND to supply negative while the servo-positive switch stays open. Plate 4 shows the pointer and D30–D32 markers at low, middle, and high.

One short safety boundary

This moving part needs an **identified servo**, its own **rated current-limited supply**, a **common ground**, and a **reachable switch or connector in servo positive**. The servo does not take power from Mega 5 V,

VIN, USB 5 V, or an I/O pin. Use only the soft paper pointer: no load, linkage, latch, spring, hard stop, or pinch point.

Choose voltage, current limit, pin order, and pulse range from the markings and primary documentation for the servo actually in hand. A kit label or visual resemblance does not turn it into a TowerPro SG90. If identity or ratings are unknown, enjoy the marker-light stage and leave servo positive disconnected.

Open servo-positive power before wiring, upload, reset, or touching the pointer. Open it immediately for unexpected motion, chatter, stall, heat, odor, supply sag, current limiting, or contact with the marked arc.

Adventure 1: make the prediction machine

Cut a light paper arrow. With every source off, place it over a flat card and draw LOW, MIDDLE, and HIGH around a generous clear arc. Do not fit it to a powered servo. The card is already useful: point the loose arrow at a label and have someone predict which marker LED should light.

Now build the three-light breadboard from plate 2. The Mega's long side header is paired: D30 sits beside D31, D32 beside D33, and D34 beside D35. Use D30, D31, and D32—not three visually adjacent holes. Each LED gets its own 220 Ω or larger resistor and returns to GND.

Part	Mega connection	Other connection	What it says
Button	D22	GND	next position
Low marker	D30	resistor, LED, GND	low requested
Middle marker	D31	resistor, LED, GND	middle requested
High marker	D32	resistor, LED, GND	high requested
Ready light	built-in D13	no wire	startup succeeded

Upload with servo positive still physically open. D13 and the middle marker appear. Each button press advances center, low, middle, and high. Use the loose paper arrow to act out each lit prediction. You now have the complete control story before anything moves.

Adventure 2: add a command signal

Turn USB off. Identify the servo connector from its own documentation, not wire color alone. Join Mega GND to supply negative first. Join D44/OC5C to servo signal second. Keep servo positive disconnected.

Net	Exact connection	Physical layout
TP-S	Mega D44/OC5C to servo signal	D44 is in the D44/D45 paired header row
Reference	Mega GND to supply negative	one named common point
Servo positive	rated supply through switch	open until Adventure 3
TP-V	servo positive to supply negative	optional clipped voltmeter

The sketch owns D44 and Timer5. It starts at the 1500 μs middle command; presses request 1500, 1000, 1500, then 2000 μs. The marker is the easy prediction. TP-S is optional deeper evidence. Neither one proves motion.

Adventure 3: let the pointer move

With all power off, secure the servo so it cannot walk across the table. Fit the light pointer only after the horn is clear of obstructions. Set the supply to the documented voltage and current limit. Make sure the physical servo-positive disconnect is within reach without crossing the pointer arc.

Power Mega logic first. The middle marker appears. Stay outside the arc, close servo positive, and let the pointer settle at center. Mark that observed position. Press through low, middle, and high, marking the observed positions only when the servo moves freely. A marker predicts a destination; the pointer is the satisfying answer.

Do not hunt mechanical endpoints. The code's bounded 1000–2000 μs range is a software range, not permission to touch a hard stop. If the exact servo requires a narrower documented range, change the compiled calibration before powering it.

> **The sketch turns its logic outputs off after two minutes. That is a tidy ending, not a power disconnect. Open servo-positive power yourself before resetting or changing the build.**

What the code is doing

The pure model maps positions 0–1000 to a reviewed pulse interval and rejects commands outside it.

```
const adk::BoundedServoConfig servoConfig =
{
    {0, 1000, 1000, 2000},
    500
};
```

The run loop remains a small story:

```
const uint16_t position = positions[nextPosition];

if (!commandPosition (position))
{
    stopLogic ();
    return;
}
```

`commandPosition()` asks `BoundedServo` for one bounded intent, writes its pulse through `ServoOutput`, and then lights the matching marker. The endpoint owns D44 and Timer5 together, so a conflict changes nothing. A partial startup failure releases earlier resources. A command failure revokes admission, stops Timer5, makes D44 high impedance, and turns off the lights.

The configuration record is checked before hardware acquisition. Its fixed-size version, length, calibration, generation, and CRC reject erased, truncated, future, corrupt, or nonsensical bytes. This lesson demonstrates the record format in memory; it does not pretend EEPROM persistence has been built.

When the picture does not match

What you see	Best next move
D13 stays dark	leave servo power open; inspect button/LED wiring and pin use
Wrong marker	compare D30/D31 and D32/D33 paired positions with plate 2
Markers work, no motion	open power; verify exact connector order, common ground, rated supply, and the open/closed disconnect
Pointer moves backward	open power; rename the card direction or use the exact servo's documented calibration; never force the horn
Buzz, chatter, heat, sag	open servo-positive power immediately; remove any obstruction and recheck identity and ratings before another run

Try one more cool thing

- Draw faces at LOW, MIDDLE, and HIGH and make a tiny mood meter.
- Make a paper dial for "sleepy / ready / excited." Keep it weightless.
- Change only the order of the four bounded positions. Predict the marker-and-pointer dance before uploading.
- In Lesson 018, the same pointer becomes an inert tabletop story prop. It never becomes a lock or a device that protects property.

For the grown-up building alongside

The hidden host suite exercises endpoint bounds, interpolation, record corruption, Timer5 and D44 conflicts, rollback, failure teardown, repeated shutdown, destruction, and resource reuse. The Mega build and size budget are also checked automatically. Those checks keep the lesson dependable without turning a family build into homework.

Current firmware size: 6480 bytes flash and 295 bytes static RAM with the repository toolchain. Physical behavior remains conditional on the exact servo and supply; no unrecorded bench result is claimed.

Useful references

- Arduino Mega 2560 documentation and pinout
- ATmega2560 Timer5 documentation
- Arduino servo external-power guidance